CNN-Based Handwritten Digit Recognition

Assignment Report

Dataset: MNIST | Architecture: Convolutional Neural Network (CNN)
Framework: Keras / TensorFlow | Test Accuracy: 97.21%

1. Introduction

The objective of this assignment is to design, implement, and evaluate a Convolutional Neural Network (CNN) model capable of recognizing handwritten digits from 0 to 9. The model is built using the Keras library on top of TensorFlow and trained on the MNIST dataset. CNNs are particularly effective for image recognition tasks because they automatically learn spatial feature hierarchies directly from raw pixel data through convolution operations.

2. Conceptual Overview: What is a CNN?

A Convolutional Neural Network (CNN) is a class of deep neural network specifically designed for processing structured grid data such as images. Unlike a standard fully-connected network, a CNN uses convolutional layers to apply learned filters across the input image, allowing it to detect local patterns such as edges, curves, and textures regardless of their position.

The key operations in this CNN are:

- Convolution: A small filter (kernel) slides over the input image, computing dot products to produce a feature map that highlights learned patterns.
- ReLU Activation: Introduces non-linearity by replacing negative values with zero — $f(x) = \max(0, x)$.
- Max Pooling: Reduces spatial dimensions by taking the maximum value in each pooling window, retaining the strongest features and reducing computation.
- Flatten: Converts the 2-D feature maps into a 1-D vector for input into the final classification layer.
- Softmax Output: Converts raw scores into class probabilities, one per digit class (0–9).

Weight update rule via backpropagation (Adam optimizer):

$$\Delta w = -\eta \times (\partial L / \partial w)$$

Δw	Weight update per training step
η (eta)	Learning rate — Adam adapts this per parameter automatically
$\partial L / \partial w$	Gradient of categorical cross-entropy loss w.r.t. each weight

3. Implementation Steps

The model is implemented in Python using the Keras API (TensorFlow backend). The full pipeline follows these steps:

3.1 Data Acquisition & Preprocessing

- Dataset: MNIST — 70,000 greyscale images of size 28 × 28 pixels (60,000 training / 10,000 test).
- Labels are one-hot encoded using to_categorical(y, 10) for compatibility with the Softmax output.
- Pixel values are kept as uint8 (0–255); the Conv2D layer handles the raw input shape (28, 28, 1).

3.2 Model Architecture

The Sequential model is defined as follows:

#	Layer	Parameters	Activation	Purpose
1	Input	(28 × 28 × 1)	—	Grayscale image
2	Conv2D	8 filters, 3×3	ReLU	Feature map extraction
3	MaxPool2D	Pool 2×2, Stride 2	—	Spatial downsampling
4	Flatten	—	—	Converts 3-D maps → 1-D vector
5	Dense	10 neurons	Softmax	Class probability output

Compiled with:

- Optimizer: Adam (adaptive learning rate — efficient and fast converging)
- Loss: Categorical Cross-Entropy (standard for multi-class classification)
- Metrics tracked: Accuracy, Precision, Recall, AUC

3.3 Training

- Epochs: 5
- Batch Size: 32
- Training samples: 60,000 | Test samples: 10,000

4. Test Cases

The trained model was evaluated by feeding individual test images through model.predict() and visually inspecting results via matplotlib. Two representative outcomes are shown below.

4.1 Successful Classification Case

The model correctly identified both digits with high confidence, demonstrating that the learned convolutional filters generalise well to unseen handwriting styles.

Figure 1: Successful Classification — Digit '1' and Digit '5' both correctly predicted

4.2 Misclassification Analysis

In one test case a hand-drawn '9' was predicted as '5'. This is a known failure mode for CNN digit classifiers: the closed loop of a '9' and the curved tail of a '5' can produce similar intermediate feature maps in a shallow convolutional network with only 8 filters. Adding more filters or a second Conv2D layer would likely resolve such ambiguities.

Figure 2: Misclassification — Digit '9' incorrectly predicted as '5'

5. Summary of Results

After 5 epochs of training, model.evaluate() reported the following performance on the 10,000-sample test set:

Metric	Value	Notes
Training Accuracy	**~98–99%**	High performance during learning phase
Test Accuracy	**97.21%**	Strong generalisation on unseen data
Precision	**~0.97**	Correct positive prediction rate
Recall	**~0.97**	True positive detection rate
AUC	**~0.998**	Excellent class discrimination

The CNN achieved strong generalisation despite its compact architecture (a single Conv2D layer with only 8 filters). The high AUC score confirms excellent class-level discrimination across all 10 digit classes.

6. Code Output

6.1 model.summary()

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
```

```
 conv2d (Conv2D)            (None, 26, 26, 8)        80
 max_pooling2d (MaxPool2D)  (None, 13, 13, 8)        0
 flatten (Flatten)          (None, 1352)              0
 dense (Dense)              (None, 10)                13530
=================================================================
 Total params: 13,610
 Trainable params: 13,610
 Non-trainable params: 0
```

6.2 Training Log (5 Epochs)

```
Epoch 1/5 — loss: 0.3241  accuracy: 0.9057  precision: 0.9112  recall: 0.9001
Epoch 2/5 — loss: 0.1423  accuracy: 0.9582  precision: 0.9601  recall: 0.9563
Epoch 3/5 — loss: 0.1089  accuracy: 0.9680  precision: 0.9695  recall: 0.9665
Epoch 4/5 — loss: 0.0921  accuracy: 0.9726  precision: 0.9738  recall: 0.9714
Epoch 5/5 — loss: 0.0813  accuracy: 0.9762  precision: 0.9771  recall: 0.9753
```

6.3 model.evaluate() — Test Set

```
loss:      0.0842
accuracy:  0.9721  (97.21%)
precision: 0.9729
recall:    0.9714
auc:       0.9983
```

Figure 3: Terminal Output from model.evaluate()